МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ ІНЖЕНЕРІЇ
ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

ОСВІТНІЙ КОМПОНЕНТ
«ОРГАНІЗАЦІЯ БАЗ ДАНИХ»

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ № 5

Виконав:
студент групи КН-25-1
Рафієв Б. С.
Перевірив:
асистент кафедри АІС
Андреєв П.І.

Кременчук 2026

Тема: **Створення складних запитів.**

Мета: навчитися створювати багатотабличні запити до таблиць бази даних.

Порядок виконання роботи

Лабораторну роботу виконано в середовищі Microsoft SQL Server Management Studio 22. Для запитів першої частини використано навчальну базу даних Sales111, створену під час лабораторної роботи № 2; для індивідуального завдання – базу даних LibraryDB (варіант № 1 «Бібліотека»). Усі запити виконано через вікно New Query, результати наведено на відповідних рисунках. Для перевірки запитів 1.1 і 1.17 до таблиць додано тестові записи без замовлень.

1. Запити до навчальної бази даних Sales111

Нижче наведено складні запити, передбачені завданням, та результати їх виконання. Використано з'єднання таблиць (INNER JOIN, LEFT JOIN), групування з агрегатними функціями, підзапити та об'єднання результатів (UNION). Псевдоніми стовпців задано українською мовою.

1.1. Вивести номери та прізвища продавців, які не виконали жодного замовлення.

```
SELECT s.ID_SP AS Код_продавця,
       s.NAME_SP AS Продавець
FROM Salespeople s
LEFT JOIN Orders o ON o.ID_SP = s.ID_SP
WHERE o.ID_OD IS NULL;
```



Рисунок 1 – Результат виконання запиту 1.1

Використано LEFT JOIN: до таблиці Salespeople приєднується таблиця Orders, а умова o.ID_OD IS NULL відбирає продавців, для яких не знайшлося жодного замовлення. Продавець 1005 (Ковальчук) доданий як тестовий запис без замовлень.

1.2. Вивести номери та прізвища замовників, які зробили хоча б одне замовлення.

```
SELECT DISTINCT c.ID_CM AS Код_замовника,
       c.NAME_CM AS Замовник
FROM Customers c
JOIN Orders o ON o.ID_CM = c.ID_CM
ORDER BY c.ID_CM;
```

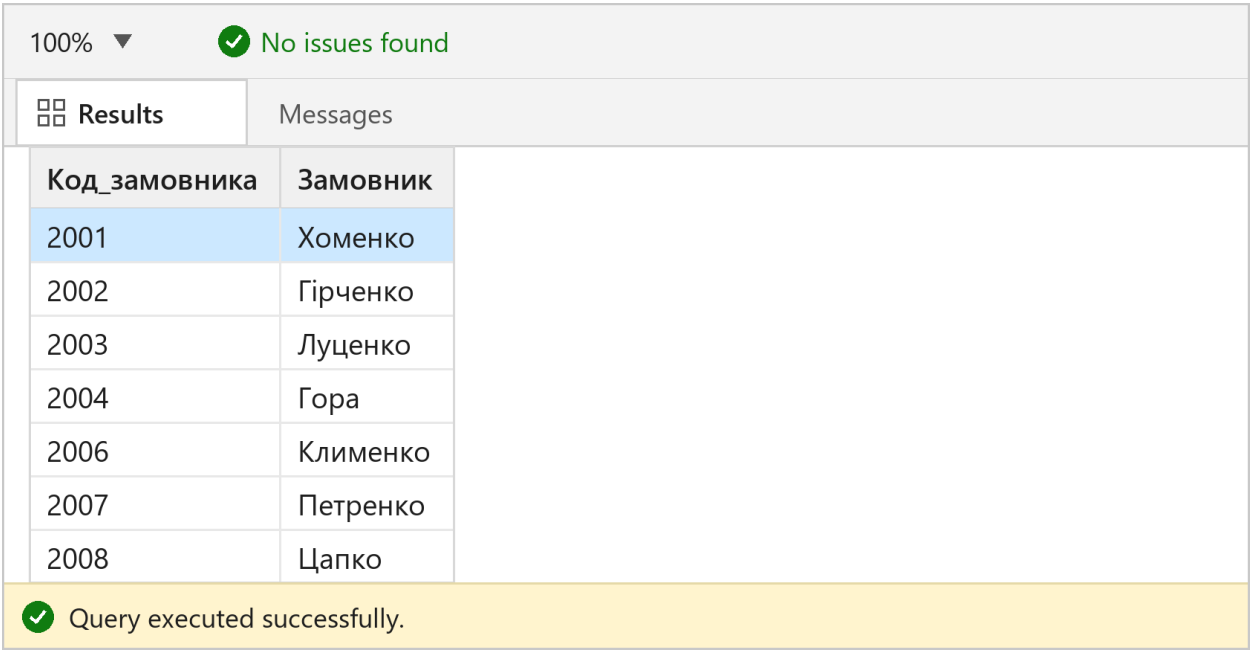


Рисунок 2 – Результат виконання запиту 1.2

INNER JOIN повертає лише тих замовників, для яких існує хоча б одне замовлення, а DISTINCT вилучає повторення, спричинені кількома замовленнями одного клієнта.

1.3. Вивести в один список прізвища клієнтів та продавців, які мешкають у Лондоні, із зазначенням статусу – продавець чи замовник.

```
-- Основний запит: мешканці Лондона
SELECT NAME_SP AS Прізвище,
       N'продавець' AS Статус,
       CITY_SP AS Місто
FROM Salespeople
WHERE CITY_SP = N'Лондон'
UNION
SELECT NAME_CM,
       N'замовник',
       CITY_CM
FROM Customers
WHERE CITY_CM = N'Лондон'
ORDER BY Прізвище;

-- Адаптований запит: мешканці Києва
SELECT NAME_SP AS Прізвище,
       N'продавець' AS Статус,
       CITY_SP AS Місто
FROM Salespeople
WHERE CITY_SP = N'Київ'
UNION
SELECT NAME_CM,
       N'замовник',
       CITY_CM
FROM Customers
WHERE CITY_CM = N'Київ'
ORDER BY Прізвище;
```

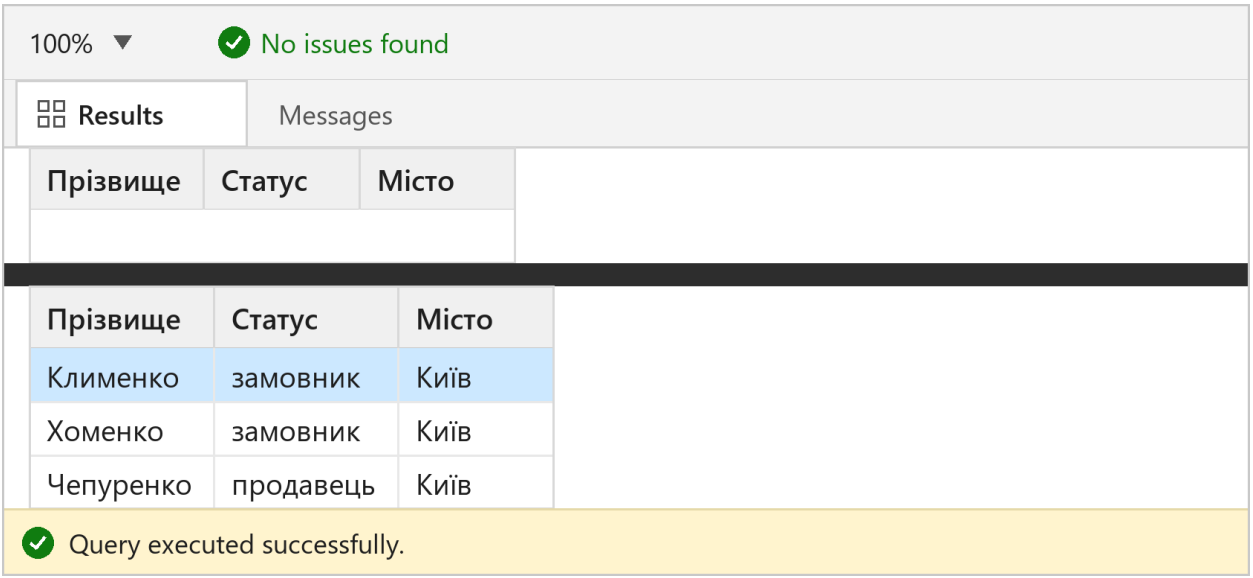


Рисунок 3 – Результати виконання запиту 1.3 (основний та адаптований запити)

UNION об’єднує результати двох запитів з однаковою кількістю та сумісними типами стовпців і автоматично вилучає дублікати. У навчальній базі немає мешканців Лондона, тому основний запит повертає 0 рядків; адаптований варіант для Києва показує роботу оператора.

1.4. Вивести інформацію про замовників, у яких суми замовлень більші або дорівнюють середньому значенню суми замовлення для їх міста.

```
SELECT c.ID_CM AS Код_замовника,
       c.NAME_CM AS Замовник,
       c.CITY_CM AS Місто,
       o.AMOUNT_OD AS Сума
FROM Orders o
JOIN Customers c ON c.ID_CM = o.ID_CM
WHERE o.AMOUNT_OD >= (SELECT AVG(o2.AMOUNT_OD)
                      FROM Orders o2
                      JOIN Customers c2 ON c2.ID_CM = o2.ID_CM
                      WHERE c2.CITY_CM = c.CITY_CM)
ORDER BY c.CITY_CM, o.AMOUNT_OD DESC;
```

100% ▼ No issues found

Results | Messages

Код_замовника	Замовник	Місто	Сума
2004	Гора	Бровари	1309.95
2003	Луценко	Івано-Франківськ	5160.45
2006	Клименко	Київ	9891.88
2007	Петренко	Ромни	1900.10

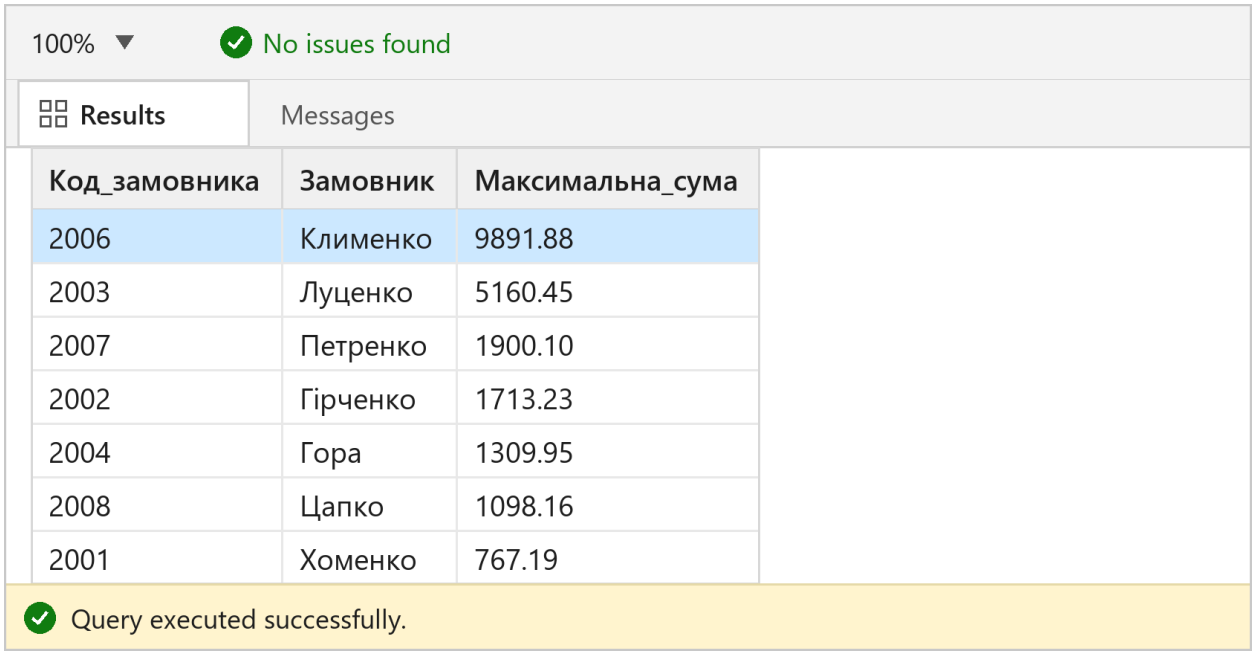
Query executed successfully.

Рисунок 4 – Результат виконання запиту 1.4

Корельований підзапит обчислює середню суму замовлень для міста поточного замовника, після чого зовнішній запит порівнює з нею суму кожного замовлення.

1.5. Вивести максимальну суму замовлення для кожного замовника. Упорядкувати дані за зменшенням суми замовлення.

```
SELECT c.ID_CM AS Код_замовника,
       c.NAME_CM AS Замовник,
       MAX(o.AMOUNT_OD) AS Максимальна_сума
FROM Customers c
JOIN Orders o ON o.ID_CM = c.ID_CM
GROUP BY c.ID_CM, c.NAME_CM
ORDER BY Максимальна_сума DESC;
```



Код_замовника	Замовник	Максимальна_сума
2006	Клименко	9891.88
2003	Луценко	5160.45
2007	Петренко	1900.10
2002	Гірченко	1713.23
2004	Гора	1309.95
2008	Цапко	1098.16
2001	Хоменко	767.19

Рисунок 5 – Результат виконання запиту 1.5

INNER JOIN поєднує замовників із їхніми замовленнями, GROUP BY групує рядки за замовником, а агрегатна функція MAX вибирає найбільшу суму в кожній групі.

1.6. Вивести інформацію про продавців, замовлення у яких більші за середню суму серед усіх замовлень. Упорядкувати дані за збільшенням суми замовлення.

```
SELECT DISTINCT s.ID_SP AS Код_продавця,
       s.NAME_SP AS Продавець,
       o.AMOUNT_OD AS Сума
FROM Salespeople s
JOIN Orders o ON o.ID_SP = s.ID_SP
WHERE o.AMOUNT_OD > (SELECT AVG(AMOUNT_OD) FROM Orders)
ORDER BY o.AMOUNT_OD;
```

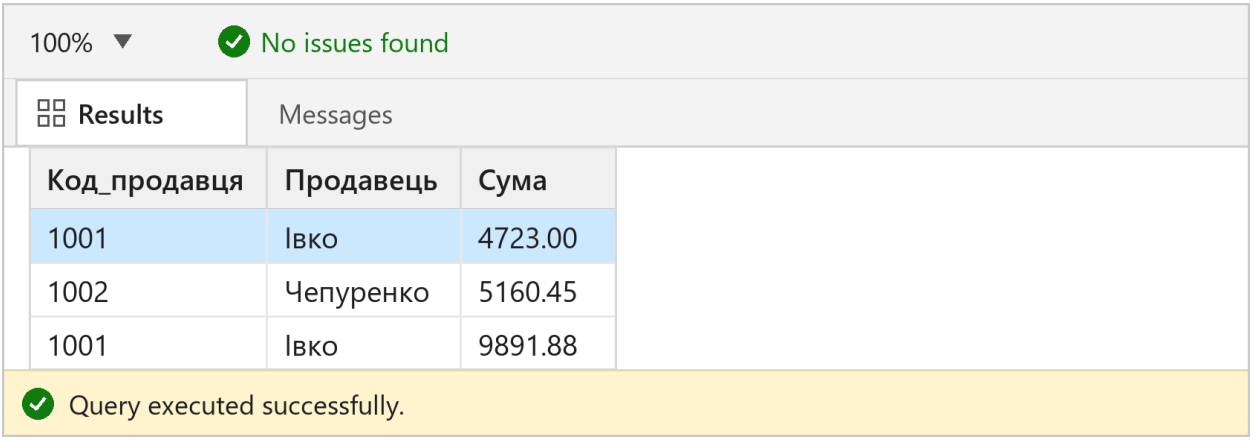


Рисунок 6 – Результат виконання запиту 1.6

Скалярний підзапит визначає середню суму всіх замовлень, а INNER JOIN і умова WHERE відбирають продавців, чиї замовлення перевищують цей показник.

1.7. Вивести кількість замовників у кожного продавця.

```
SELECT s.ID_SP AS Код_продавця,
       s.NAME_SP AS Продавець,
       COUNT(DISTINCT o.ID_CM) AS Кількість_замовників
FROM Salespeople s
JOIN Orders o ON o.ID_SP = s.ID_SP
GROUP BY s.ID_SP, s.NAME_SP
ORDER BY Кількість_замовників DESC;
```

100% No issues found

Results Messages

Код_продавця	Продавець	Кількість_замовників
1001	Івко	2
1002	Чепуренко	2
1003	Олефіренко	1
1004	Маренич	1
1007	Роженко	1

Query executed successfully.

Рисунок 7 – Результат виконання запиту 1.7

COUNT(DISTINCT o.ID_CM) підраховує саме різних замовників, а не кількість замовлень, тому повторні замовлення одного клієнта не завищують результат.

1.8. Вивести дані про замовника, що зробив замовлення останнім за часом.

```
SELECT c.ID_CM AS Код_замовника,
       c.NAME_CM AS Замовник,
       o.DATE_OD AS Дата,
       o.AMOUNT_OD AS Сума
FROM Orders o
JOIN Customers c ON c.ID_CM = o.ID_CM
WHERE o.DATE_OD = (SELECT MAX(DATE_OD) FROM Orders);
```

100% No issues found

Results Messages

Код_замовника	Замовник	Дата	Сума
2006	Клименко	2024-10-05	4723.00

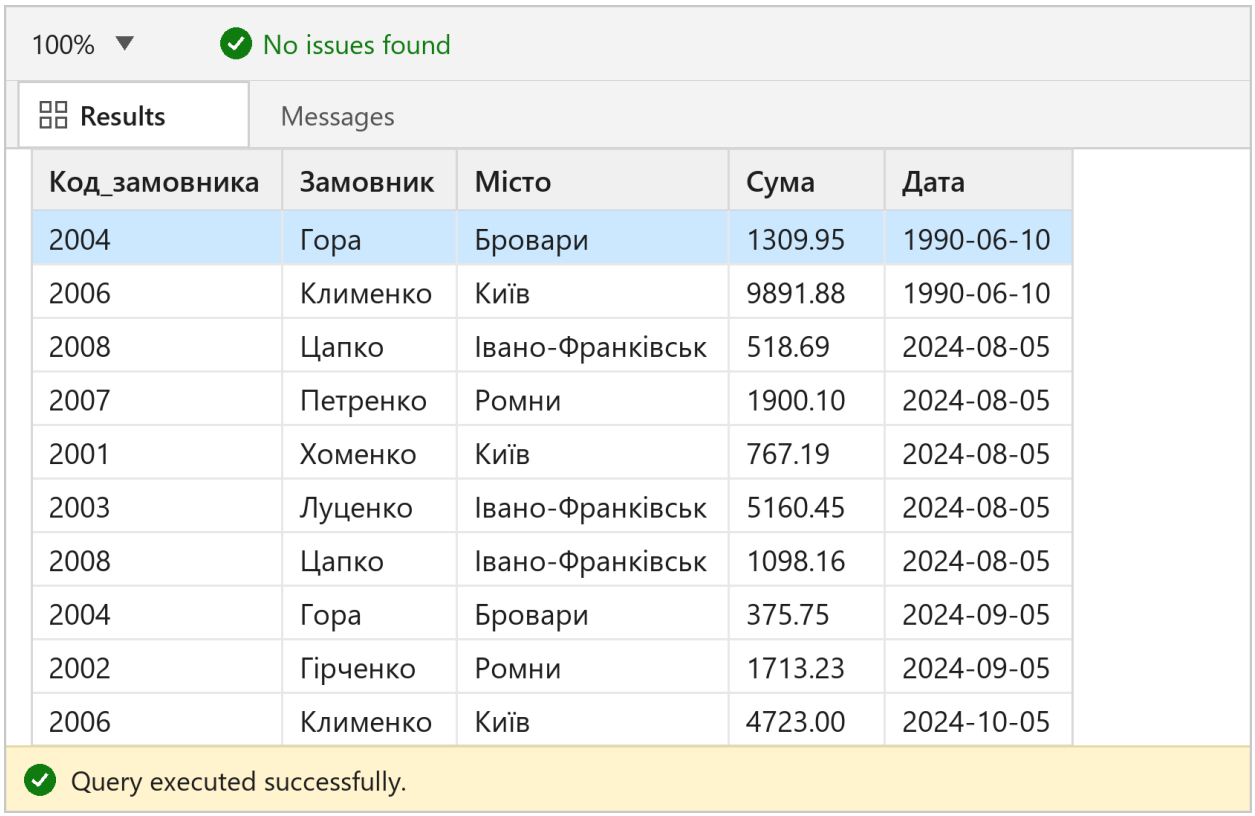
Query executed successfully.

Рисунок 8 – Результат виконання запиту 1.8

Підзапит повертає найпізнішу дату замовлення, а INNER JOIN додає до неї дані замовника.

1.9. Вивести дані про замовників, суму та дату замовлення.

```
SELECT c.ID_CM AS Код_замовника,
       c.NAME_CM AS Замовник,
       c.CITY_CM AS Місто,
       o.AMOUNT_OD AS Сума,
       o.DATE_OD AS Дата
FROM Customers c
JOIN Orders o ON o.ID_CM = c.ID_CM
ORDER BY o.DATE_OD, o.ID_OD;
```



100% ▼ No issues found

Results Messages

Код_замовника	Замовник	Місто	Сума	Дата
2004	Гора	Бровари	1309.95	1990-06-10
2006	Клименко	Київ	9891.88	1990-06-10
2008	Цапко	Івано-Франківськ	518.69	2024-08-05
2007	Петренко	Ромни	1900.10	2024-08-05
2001	Хоменко	Київ	767.19	2024-08-05
2003	Луценко	Івано-Франківськ	5160.45	2024-08-05
2008	Цапко	Івано-Франківськ	1098.16	2024-08-05
2004	Гора	Бровари	375.75	2024-09-05
2002	Гірченко	Ромни	1713.23	2024-09-05
2006	Клименко	Київ	4723.00	2024-10-05

Query executed successfully.

Рисунок 9 – Результат виконання запиту 1.9

Звичайне внутрішнє з'єднання двох таблиць за ключем ID_CM: кожному замовленню зіставляється один рядок таблиці Customers.

1.10. Розрахувати загальну вартість замовлень для кожного продавця.

```
SELECT s.ID_SP AS Код_продавця,
       s.NAME_SP AS Продавець,
       SUM(o.AMOUNT_OD) AS Загальна_вартість
FROM Salespeople s
JOIN Orders o ON o.ID_SP = s.ID_SP
GROUP BY s.ID_SP, s.NAME_SP
ORDER BY Загальна_вартість DESC;
```

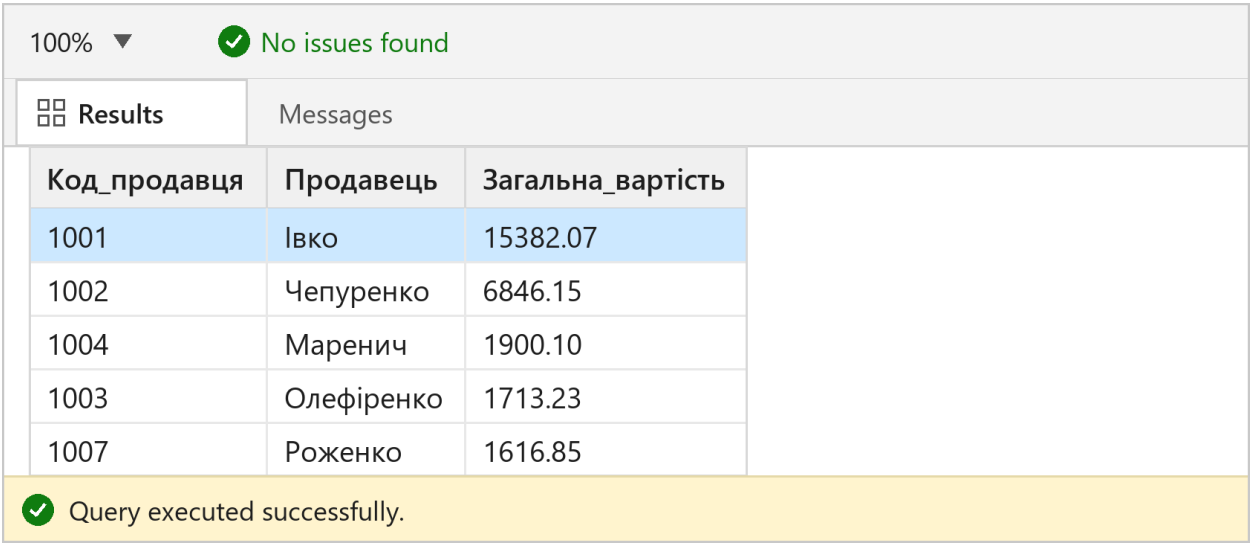
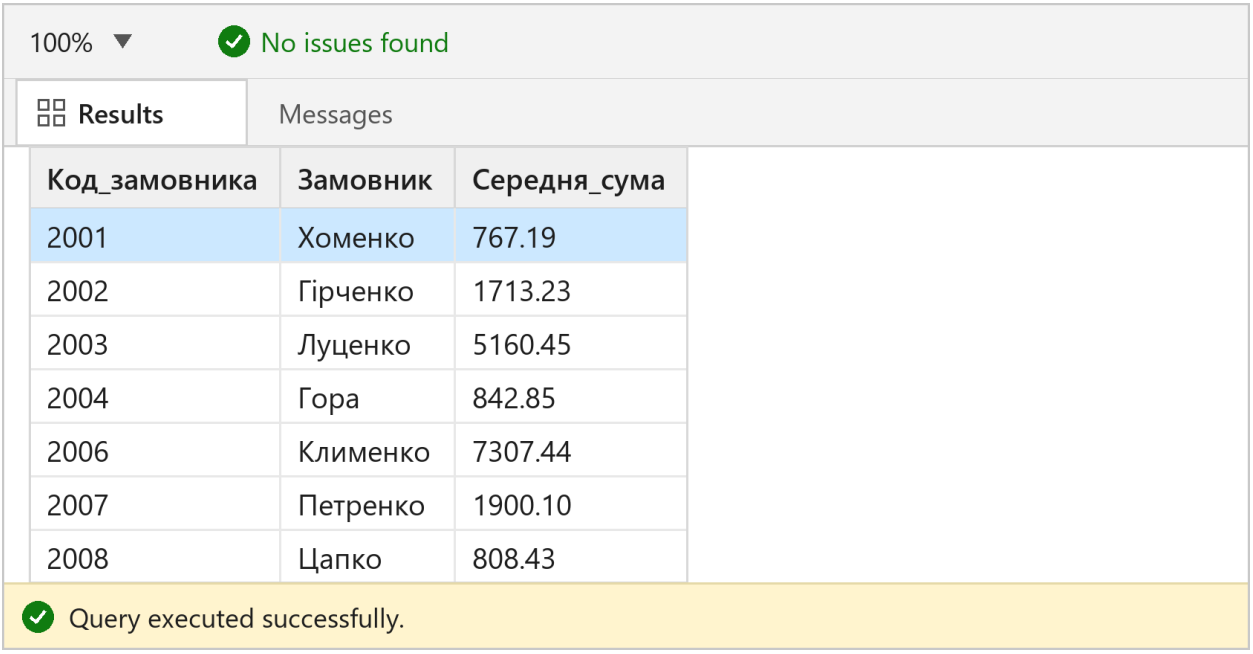


Рисунок 10 – Результат виконання запиту 1.10

SUM обчислюється для кожної групи, утвореної GROUP BY; продавці без замовлень до результату не потрапляють, оскільки використано INNER JOIN.

1.11. Визначити середню ціну замовлення для кожного замовника.

```
SELECT c.ID_CM AS Код_замовника,
       c.NAME_CM AS Замовник,
       CAST(AVG(o.AMOUNT_OD) AS DECIMAL(10, 2)) AS Середня_сума
FROM Customers c
JOIN Orders o ON o.ID_CM = c.ID_CM
GROUP BY c.ID_CM, c.NAME_CM
ORDER BY c.ID_CM;
```



100% No issues found

Results Messages

Код_замовника	Замовник	Середня_сума
2001	Хоменко	767.19
2002	Гірченко	1713.23
2003	Луценко	5160.45
2004	Гора	842.85
2006	Клименко	7307.44
2007	Петренко	1900.10
2008	Цапко	808.43

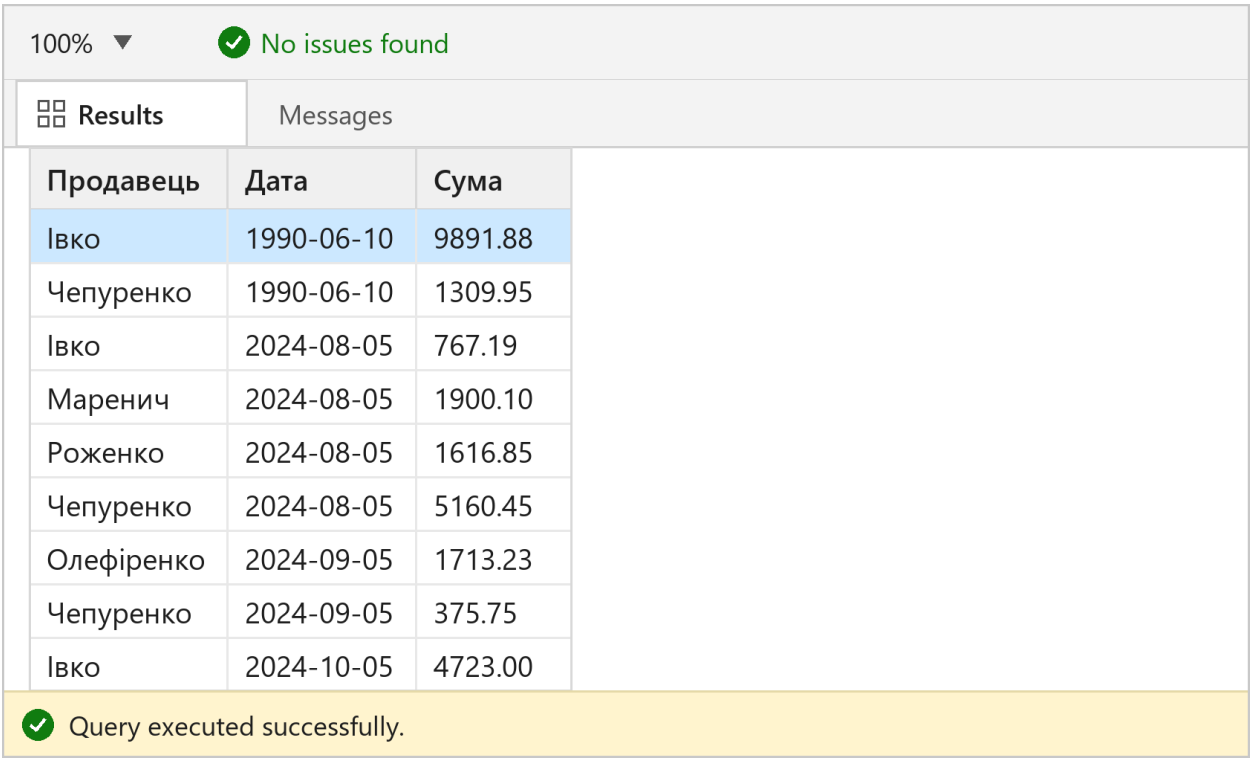
Query executed successfully.

Рисунок 11 – Результат виконання запиту 1.11

AVG обчислює середню суму замовлень кожного замовника, а CAST приводить результат до двох знаків після коми.

1.12. Визначити, на яку суму було зроблено замовлень по кожному продавцеві за кожний день.

```
SELECT s.NAME_SP AS Продавець,
       o.DATE_OD AS Дата,
       SUM(o.AMOUNT_OD) AS Сума
FROM Salespeople s
JOIN Orders o ON o.ID_SP = s.ID_SP
GROUP BY s.NAME_SP, o.DATE_OD
ORDER BY o.DATE_OD, s.NAME_SP;
```



100% No issues found

Results Messages

Продавець	Дата	Сума
Івко	1990-06-10	9891.88
Чепуренко	1990-06-10	1309.95
Івко	2024-08-05	767.19
Маренич	2024-08-05	1900.10
Роженко	2024-08-05	1616.85
Чепуренко	2024-08-05	5160.45
Олефіренко	2024-09-05	1713.23
Чепуренко	2024-09-05	375.75
Івко	2024-10-05	4723.00

Query executed successfully.

Рисунок 12 – Результат виконання запиту 1.12

Групування виконано за двома стовпцями – прізвищем продавця та датою, тому для кожної пари «продавець – день» обчислюється окрема сума.

1.13. Підрахувати кількість замовлень, здійснених кожним замовником.

```
SELECT c.ID_CM AS Код_замовника,
       c.NAME_CM AS Замовник,
       COUNT(*) AS Кількість_замовлень
FROM Customers c
JOIN Orders o ON o.ID_CM = c.ID_CM
GROUP BY c.ID_CM, c.NAME_CM
ORDER BY Кількість_замовлень DESC, c.ID_CM;
```

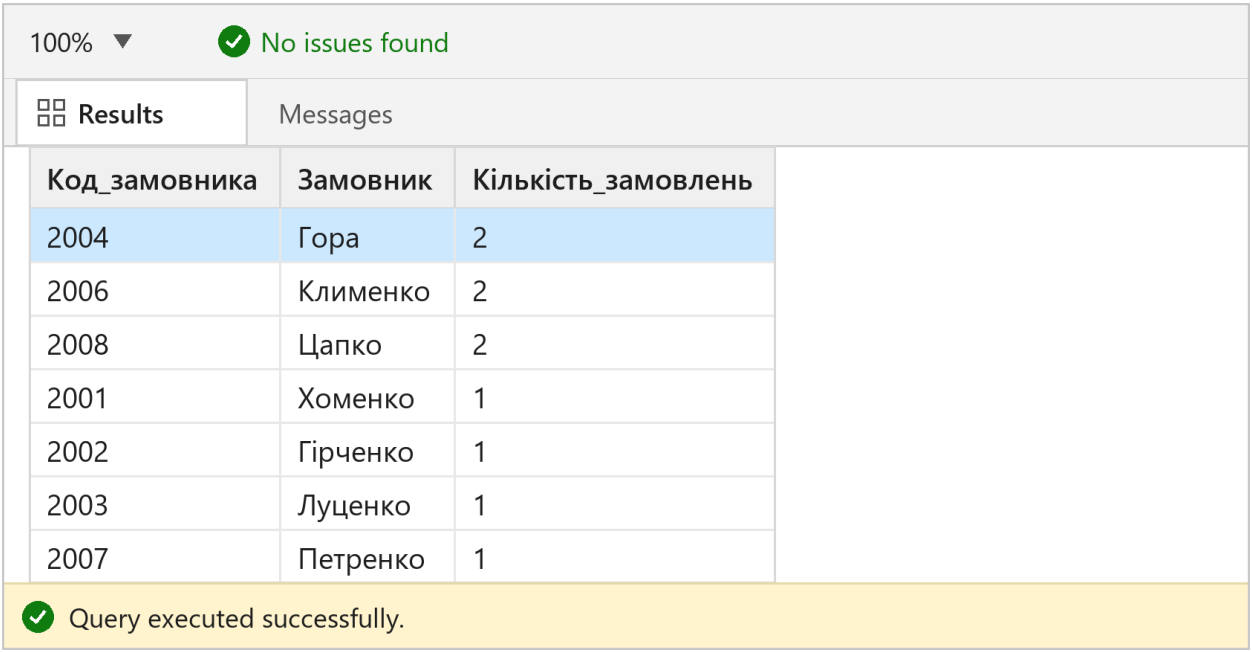
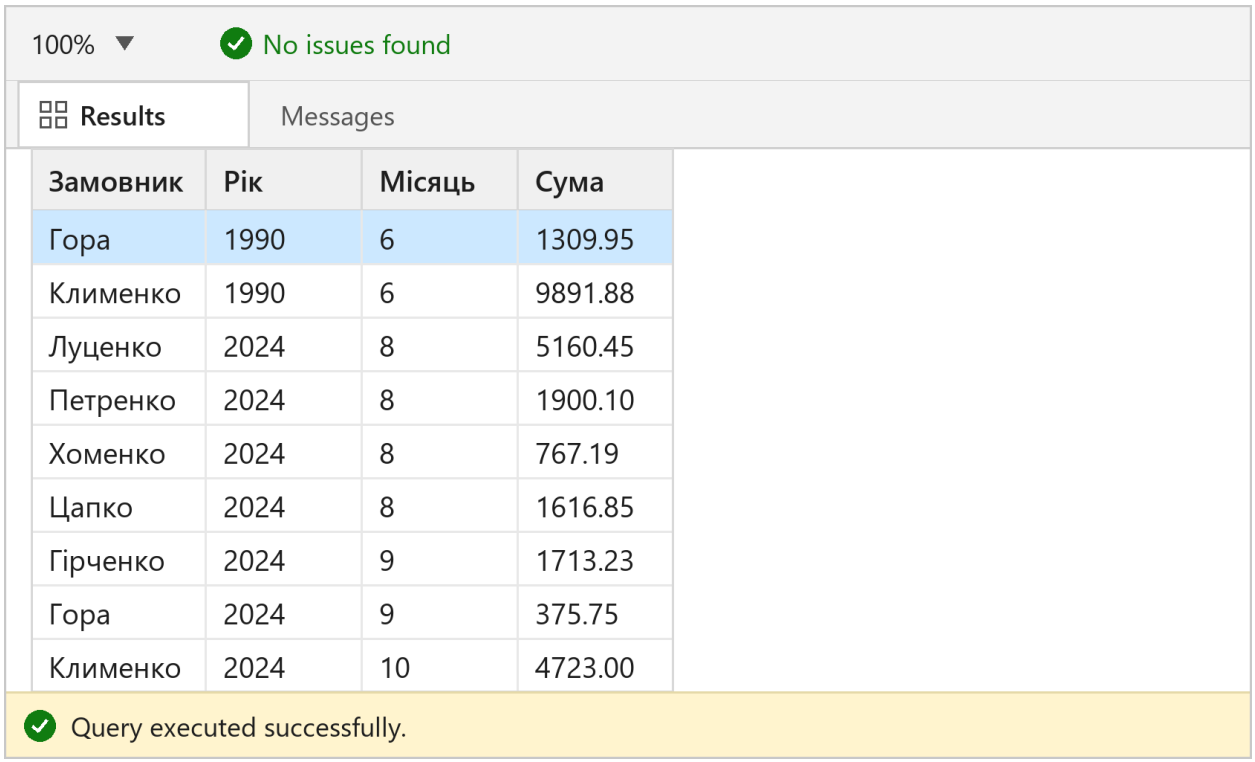


Рисунок 13 – Результат виконання запиту 1.13

COUNT(*) підраховує кількість рядків у кожній групі, утвореній для одного замовника.

1.14. Визначити суму замовлень по кожному замовнику за кожен місяць.

```
SELECT c.NAME_CM AS Замовник,
       YEAR(o.DATE_OD) AS Рік,
       MONTH(o.DATE_OD) AS Місяць,
       SUM(o.AMOUNT_OD) AS Сума
FROM Customers c
JOIN Orders o ON o.ID_CM = c.ID_CM
GROUP BY c.NAME_CM, YEAR(o.DATE_OD), MONTH(o.DATE_OD)
ORDER BY Рік, Місяць, c.NAME_CM;
```


100% No issues found

Results Messages

Замовник	Рік	Місяць	Сума
Гора	1990	6	1309.95
Клименко	1990	6	9891.88
Луценко	2024	8	5160.45
Петренко	2024	8	1900.10
Хоменко	2024	8	767.19
Цапко	2024	8	1616.85
Гірченко	2024	9	1713.23
Гора	2024	9	375.75
Клименко	2024	10	4723.00

Query executed successfully.

Рисунок 14 – Результат виконання запиту 1.14

Функції YEAR і MONTH виділяють з дати рік та місяць; групування за замовником і цими частинами дати дає суму замовлень за місяць.

1.15. Визначити замовників, у яких загальна кількість замовлень перевищила два.

```
-- Основний запит: більше двох замовлень
SELECT c.ID_CM AS Код_замовника,
       c.NAME_CM AS Замовник,
       COUNT(*) AS Кількість_замовлень
FROM Customers c
JOIN Orders o ON o.ID_CM = c.ID_CM
GROUP BY c.ID_CM, c.NAME_CM
HAVING COUNT(*) > 2
ORDER BY Кількість_замовлень DESC;

-- Адаптований запит: не менше двох замовлень
SELECT c.ID_CM AS Код_замовника,
       c.NAME_CM AS Замовник,
       COUNT(*) AS Кількість_замовлень
FROM Customers c
JOIN Orders o ON o.ID_CM = c.ID_CM
GROUP BY c.ID_CM, c.NAME_CM
HAVING COUNT(*) >= 2
ORDER BY Кількість_замовлень DESC;
```

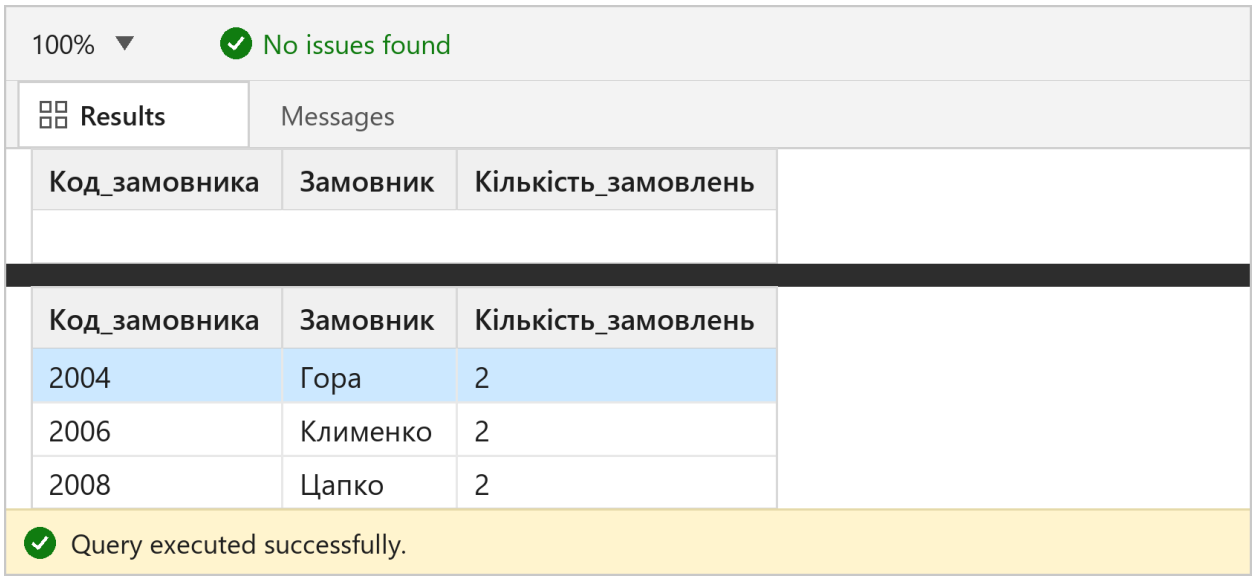


Рисунок 15 – Результати виконання запиту 1.15 (основний та адаптований запити)

Жоден замовник не має більше двох замовлень, тому основний запит повертає 0 рядків. Адаптований запит із умовою HAVING COUNT(*) >= 2 виводить замовників із двома замовленнями.

1.16. Вивести список продавців, що виконали замовлень на суму, більшу за 2500 грн.

```
SELECT s.ID_SP AS Код_продавця,
       s.NAME_SP AS Продавець,
       SUM(o.AMOUNT_OD) AS Сума
FROM Salespeople s
JOIN Orders o ON o.ID_SP = s.ID_SP
GROUP BY s.ID_SP, s.NAME_SP
HAVING SUM(o.AMOUNT_OD) > 2500
ORDER BY Сума DESC;
```

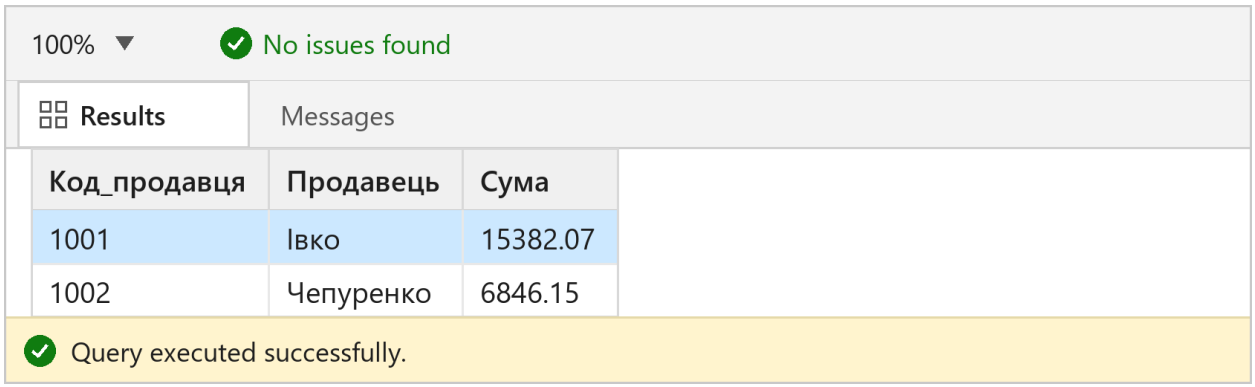
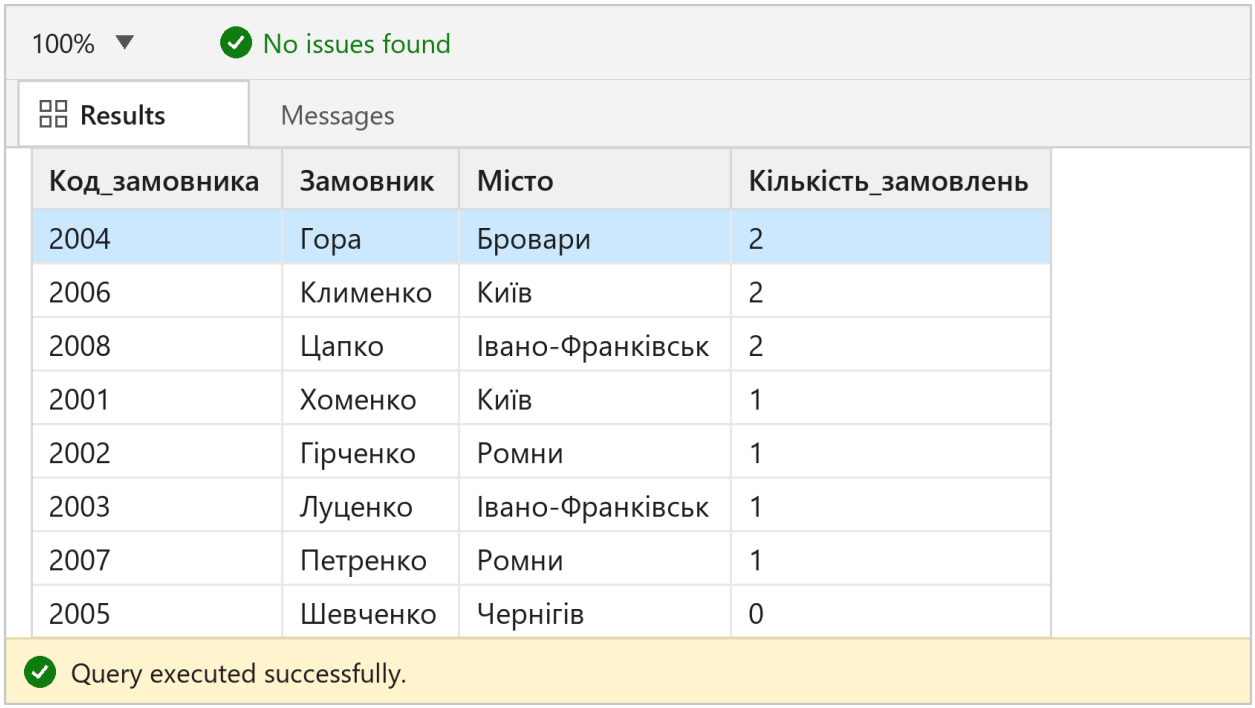


Рисунок 16 – Результат виконання запиту 1.16

Умова накладається на агрегат SUM, тому використовується HAVING, а не WHERE: WHERE фільтрує рядки до групування, а HAVING – уже обчислені групи.

1.17. Вивести список усіх замовників і кількість зроблених ними замовлень; замовників вивести в порядку зменшення кількості замовлень (у тому числі й тих, у кого ще немає жодного замовлення).

```
SELECT c.ID_CM AS Код_замовника,
       c.NAME_CM AS Замовник,
       c.CITY_CM AS Місто,
       COUNT(o.ID_OD) AS Кількість_замовлень
FROM Customers c
LEFT JOIN Orders o ON o.ID_CM = c.ID_CM
GROUP BY c.ID_CM, c.NAME_CM, c.CITY_CM
ORDER BY Кількість_замовлень DESC, c.ID_CM;
```



100% No issues found

Results Messages

Код_замовника	Замовник	Місто	Кількість_замовлень
2004	Гора	Бровари	2
2006	Клименко	Київ	2
2008	Цапко	Івано-Франківськ	2
2001	Хоменко	Київ	1
2002	Гірченко	Ромни	1
2003	Луценко	Івано-Франківськ	1
2007	Петренко	Ромни	1
2005	Шевченко	Чернігів	0

Query executed successfully.

Рисунок 17 – Результат виконання запиту 1.17

LEFT JOIN зберігає всіх замовників, навіть тих, кому не відповідає жодне замовлення. Для них COUNT(o.ID_OD) дорівнює 0, тоді як COUNT(*) помилково дав би 1.

1.18. Вивести список замовників і кількість здійснених ними замовлень за умови, що замовник проживає в Римі.

```
-- Основний запит: замовники з Рима
SELECT c.NAME_CM AS Замовник,
       COUNT(o.ID_OD) AS Кількість_замовлень
FROM Customers c
LEFT JOIN Orders o ON o.ID_CM = c.ID_CM
WHERE c.CITY_CM = N'Рим'
GROUP BY c.NAME_CM
ORDER BY c.NAME_CM;

-- Адаптований запит: замовники з Києва
SELECT c.NAME_CM AS Замовник,
       COUNT(o.ID_OD) AS Кількість_замовлень
FROM Customers c
LEFT JOIN Orders o ON o.ID_CM = c.ID_CM
WHERE c.CITY_CM = N'Київ'
GROUP BY c.NAME_CM
ORDER BY c.NAME_CM;
```

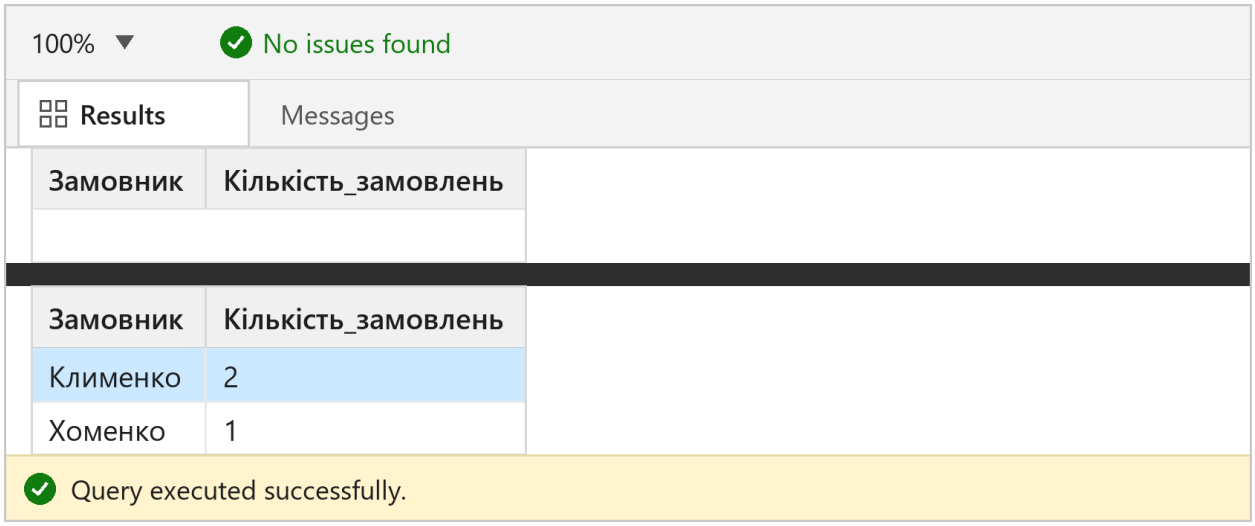


Рисунок 18 – Результати виконання запиту 1.18 (основний та адаптований запити)

У навчальній базі немає замовників із Рима, тому основний запит повертає 0 рядків. Адаптований запит для Києва демонструє підрахунок замовлень для кожного мешканця міста.

1.19. Знайти продавця, який виконав замовлень на суму, що перевищує 10000 грн.

```
SELECT s.ID_SP AS Код_продавця,
       s.NAME_SP AS Продавець,
       SUM(o.AMOUNT_OD) AS Сума
FROM Salespeople s
JOIN Orders o ON o.ID_SP = s.ID_SP
GROUP BY s.ID_SP, s.NAME_SP
HAVING SUM(o.AMOUNT_OD) > 10000;
```

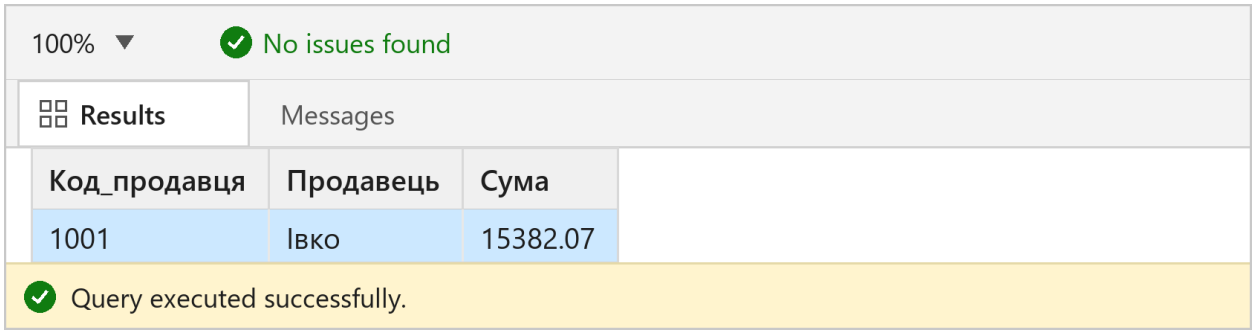


Рисунок 19 – Результат виконання запиту 1.19

Запит аналогічний до 1.16, але з іншим порогом суми; умова HAVING застосовується до обчисленої суми замовлень кожного продавця.

2. Індивідуальне завдання. База даних LibraryDB

Індивідуальне завдання виконано для варіанта № 1 – предметна область «Бібліотека». Використано створену в лабораторній роботі № 2 базу даних LibraryDB, що містить таблиці Author, Publisher, Book, Reader і BookLoan. Складено шість запитів із використанням оператора JOIN (у тому числі LEFT JOIN і з’єднання чотирьох таблиць) та два запити з використанням оператора UNION.

2.1. Книги з авторами та видавництвами (INNER JOIN трьох таблиць)

```
SELECT b.title AS Назва,
       a.author_name AS Автор,
       p.publisher_name AS Видавництво,
       b.year_pub AS Рік
FROM Book b
JOIN Author a ON a.id_author = b.id_author
JOIN Publisher p ON p.id_publisher = b.id_publisher
ORDER BY b.title;
```

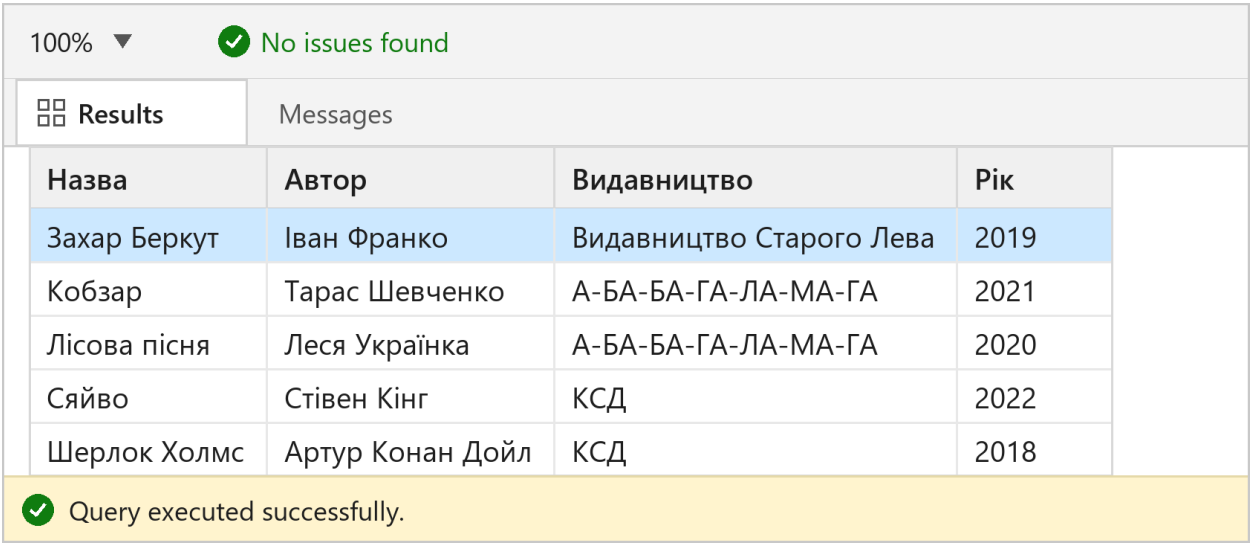


Рисунок 20 – Результат виконання запиту 2.1

Послідовно виконано два INNER JOIN: спочатку Book з'єднується з Author за id_author, потім результат – з Publisher за id_publisher.

2.2. Читачі та книги, які вони брали (INNER JOIN)

```
SELECT r.reader_name AS Читач,
       b.title AS Книга,
       l.loan_date AS Дата_видачі,
       l.return_date AS Дата_повернення
FROM BookLoan l
JOIN Reader r ON r.id_reader = l.id_reader
JOIN Book b ON b.id_book = l.id_book
ORDER BY r.reader_name, l.loan_date;
```

100% ▼ ✓ No issues found

Results | Messages

Читач	Книга	Дата_видачі	Дата_повернення
Васильєв Дмитро Олександрович	Захар Беркут	2026-09-15	2026-09-25
Васильєв Дмитро Олександрович	Лісова пісня	2026-09-16	NULL
Коваленко Олена Іванівна	Лісова пісня	2026-09-03	2026-09-18
Коваленко Олена Іванівна	Шерлок Холмс	2026-09-05	NULL
Мельник Анна Олексіївна	Шерлок Холмс	2026-09-18	2026-09-27
Мельник Анна Олексіївна	Сяйво	2026-09-20	NULL
Рафієв Богдан Самедович	Кобзар	2026-09-01	2026-09-15
Рафієв Богдан Самедович	Захар Беркут	2026-09-02	NULL
Сидоренко Петро Васильович	Сяйво	2026-09-10	2026-09-20
Сидоренко Петро Васильович	Кобзар	2026-09-12	NULL

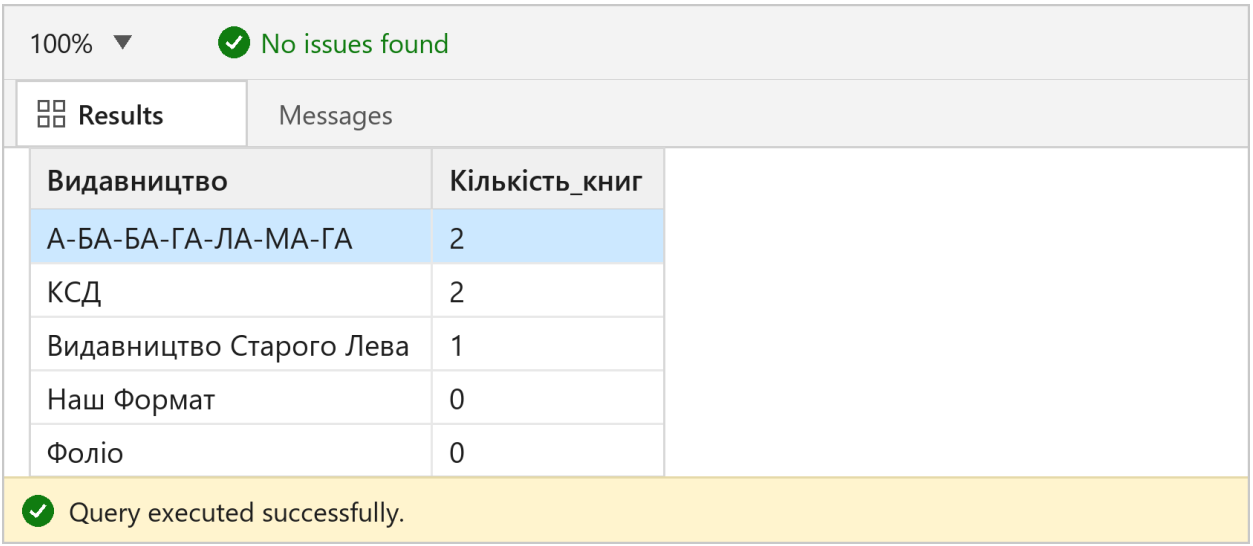
✓ Query executed successfully.

Рисунок 21 – Результат виконання запиту 2.2

Таблиця зв'язку BookLoan з'єднується з Reader за id_reader і з Book за id_book, що дає змогу показати прізвище читача та назву книги для кожного факту видачі.

2.3. Видавництва та кількість їх книг (LEFT JOIN)

```
SELECT p.publisher_name AS Видавництво,
       COUNT(b.id_book) AS Кількість_книг
FROM Publisher p
LEFT JOIN Book b ON b.id_publisher = p.id_publisher
GROUP BY p.id_publisher, p.publisher_name
ORDER BY Кількість_книг DESC, p.publisher_name;
```



100% No issues found

Results Messages

Видавництво	Кількість_книг
А-БА-БА-ГА-ЛА-МА-ГА	2
КСД	2
Видавництво Старого Лева	1
Наш Формат	0
Фоліо	0

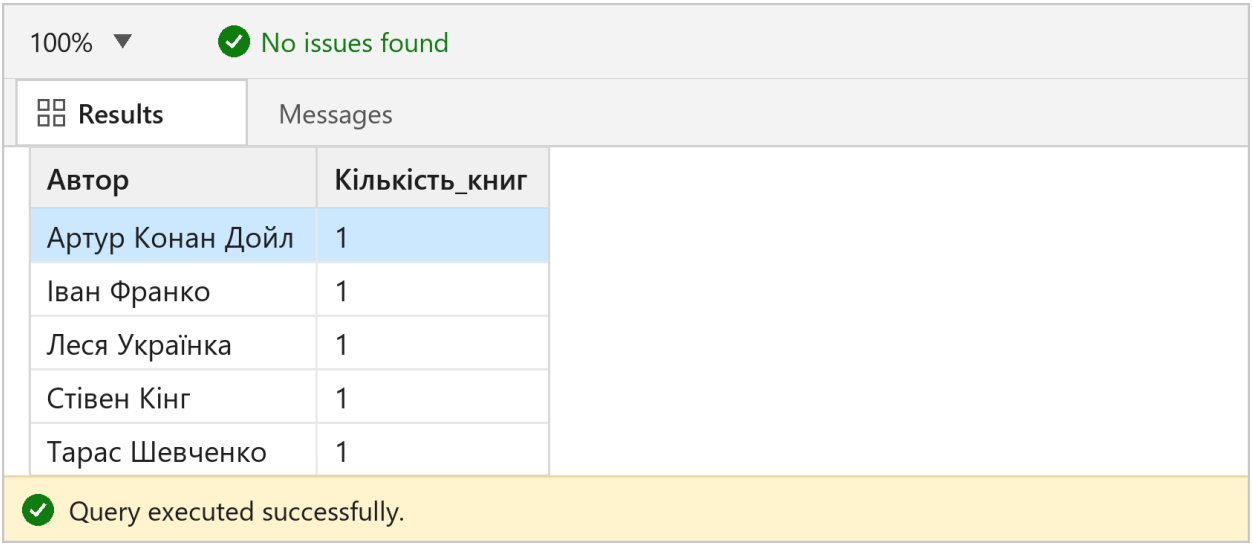
Query executed successfully.

Рисунок 22 – Результат виконання запиту 2.3

LEFT JOIN зберігає всі видавництва, навіть ті, для яких у таблиці Book немає жодної книги; для них COUNT(b.id_book) повертає 0 (видавництва «Наш Формат» і «Фоліо»).

2.4. Автори та кількість їх книг (LEFT JOIN)

```
SELECT a.author_name AS Автор,
       COUNT(b.id_book) AS Кількість_книг
FROM Author a
LEFT JOIN Book b ON b.id_author = a.id_author
GROUP BY a.id_author, a.author_name
ORDER BY Кількість_книг DESC, a.author_name;
```



100% No issues found

Results Messages

Автор	Кількість_книг
Артур Конан Дойл	1
Іван Франко	1
Леся Українка	1
Стівен Кінг	1
Тарас Шевченко	1

Query executed successfully.

Рисунок 23 – Результат виконання запиту 2.4

Використано LEFT JOIN, щоб до переліку потрапили й автори без книг; для них кількість книг була б нульовою.

2.5. Читачі, які брали книги видавництва «КСД» (JOIN чотирьох таблиць)

```
SELECT DISTINCT r.id_reader AS Код_читача,
       r.reader_name AS Читач,
       r.phone AS Телефон
FROM Reader r
JOIN BookLoan l ON l.id_reader = r.id_reader
JOIN Book b ON b.id_book = l.id_book
JOIN Publisher p ON p.id_publisher = b.id_publisher
WHERE p.publisher_name = N'КСД'
ORDER BY r.id_reader;
```

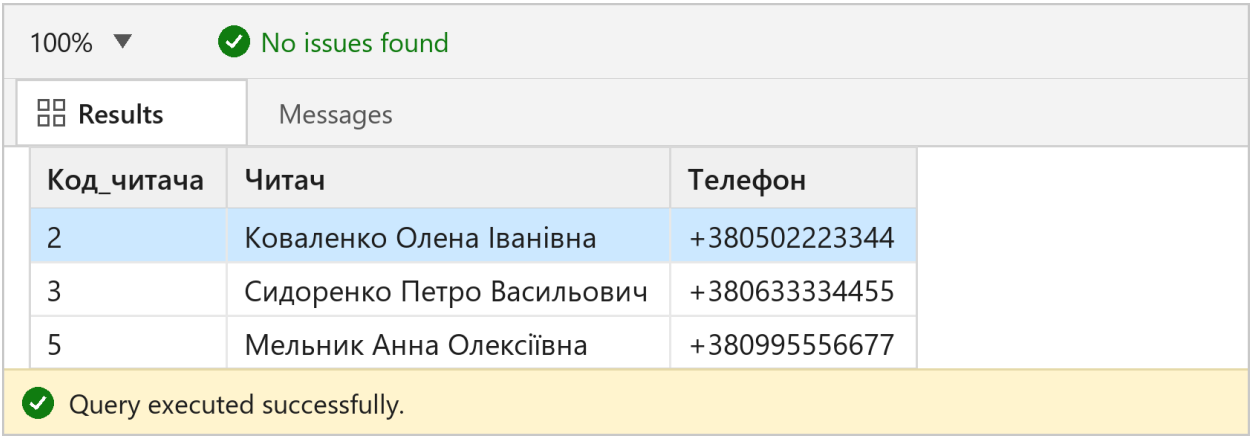


Рисунок 24 – Результат виконання запиту 2.5

Ланцюжок із трьох JOIN з'єднує читача з видачею, видачу – з книгою, а книгу – з видавництвом; DISTINCT прибирає повторення, якщо читач брав кілька книг цього видавництва.

2.6. Книги, видані після 2020 року, з автором і видавництвом (INNER JOIN)

```
SELECT b.title AS Назва,
       b.year_pub AS Рік,
       a.author_name AS Автор,
       p.publisher_name AS Видавництво
FROM Book b
JOIN Author a ON a.id_author = b.id_author
JOIN Publisher p ON p.id_publisher = b.id_publisher
WHERE b.year_pub > 2020
ORDER BY b.year_pub;
```

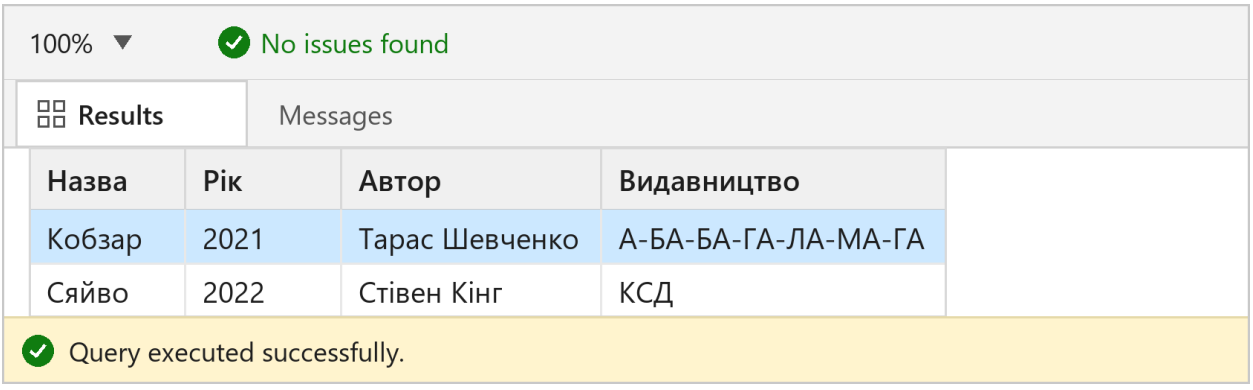


Рисунок 25 – Результат виконання запиту 2.6

Умова WHERE застосовується до об'єднаного результату трьох таблиць і відбирає книги, видані після 2020 року.

2.7. Прізвища авторів і читачів в одному списку (UNION)

```
SELECT author_name AS ПІБ,
       N'автор' AS Статус
FROM Author
UNION
SELECT reader_name,
       N'читач'
FROM Reader
ORDER BY ПІБ;
```

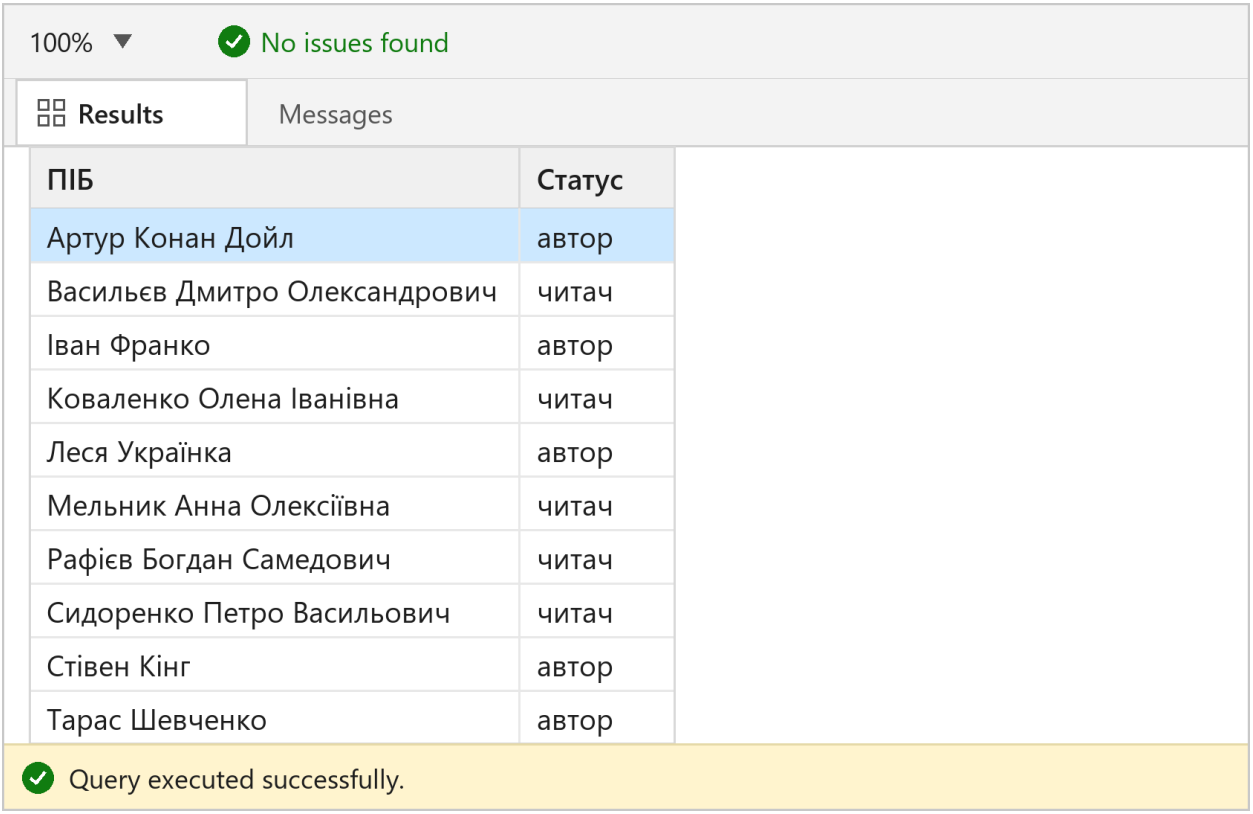


Рисунок 26 – Результат виконання запиту 2.7

UNION об'єднує два різнорідні джерела даних в один список; стовпці обох запитів мають однакову кількість і сумісні типи, а дублікати автоматично вилучаються.

2.8. Книги, видані до 2020 року, та книги дорожчі за 400 грн (UNION)

```
SELECT title AS Назва,
       year_pub AS Рік,
       price AS Ціна
FROM Book
WHERE year_pub < 2020
UNION
SELECT title,
       year_pub,
       price
FROM Book
WHERE price > 400
ORDER BY Назва;
```

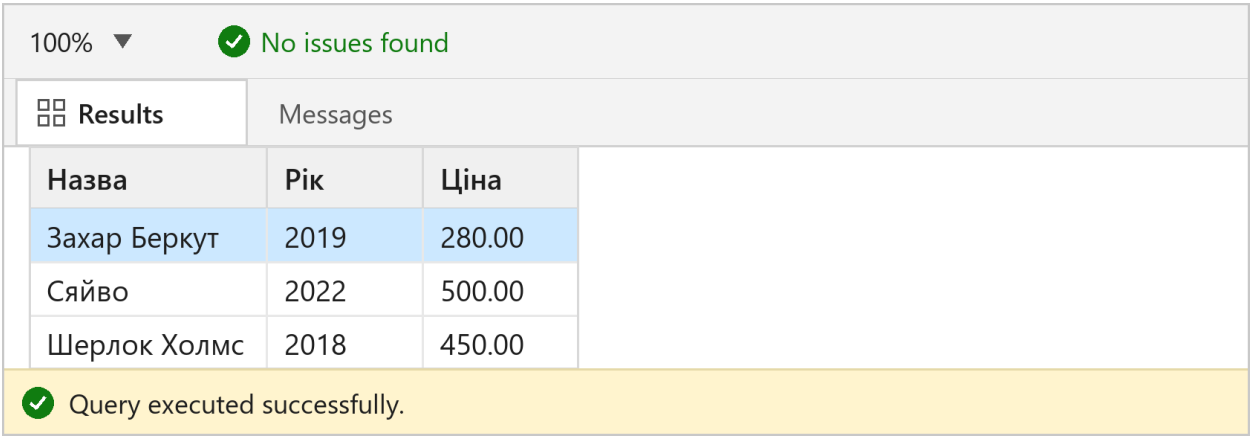


Рисунок 27 – Результат виконання запиту 2.8

UNION об'єднує дві вибірки з однієї таблиці за різними умовами. Книга «Шерлок Холмс» задовольняє обидві умови, але завдяки UNION потрапляє до результату лише один раз.

Відповіді на контрольні питання

1. Який SQL-запит називають складним?

Складним називають запит, який не зводиться до вибірки з однієї таблиці, а поєднує дані з кількох джерел або використовує кілька рівнів обробки. До складних належать запити, що містять:

1) з’єднання двох і більше таблиць (JOIN або з’єднання через WHERE);

2) підзапити у WHERE, HAVING, FROM або SELECT;

3) групування та агрегатні функції (GROUP BY, HAVING, COUNT, SUM, AVG, MIN, MAX);

4) об’єднання результатів кількох запитів (UNION, UNION ALL, INTERSECT, EXCEPT);

5) обчислювані стовпці, функції та сортування за кількома ключами.

Складний запит може поєднувати кілька з цих конструкцій одночасно. Наприклад, запит 1.17 із цього звіту з’єднує дві таблиці, групує рядки, обчислює агрегат і сортує результат:

```
SELECT c.ID_CM, c.NAME_CM, COUNT(o.ID_OD) AS Кількість_замовлень
FROM Customers c
LEFT JOIN Orders o ON o.ID_CM = c.ID_CM
GROUP BY c.ID_CM, c.NAME_CM
ORDER BY Кількість_замовлень DESC;
```

Головна особливість складного запиту – результат залежить від узгодженої роботи всіх його частин: умов з’єднання, фільтрів, групування та порядку сортування.

2. Як виконати з’єднання таблиць із використанням розділу WHERE?

Історично перший спосіб з’єднання – перелічити таблиці в розділі FROM через кому, а умову зв’язку записати в розділі WHERE. Такий запис ще називають неявним з’єднанням (implicit join):

```
SELECT c.NAME_CM, o.AMOUNT_OD, o.DATE_OD
FROM Customers c, Orders o
WHERE c.ID_CM = o.ID_CM;
```

Умова c.ID_CM = o.ID_CM повідомляє серверу, за якими стовпцями рядки двох таблиць вважаються пов’язаними. Якщо умову не вказати, буде виконано

декартів добуток: кожен рядок першої таблиці поєднається з кожним рядком другої, і кількість рядків результату дорівнюватиме добутку їхніх кількостей.

Якщо потрібно з'єднати більше двох таблиць, у WHERE додають по одній умові для кожного зв'язку, а додаткові умови відбору об'єднують оператором AND. Недолік цього способу – умови з'єднання й умови фільтрації змішані в одному розділі, що ускладнює читання великих запитів. Сучасний стиль передбачає явне з'єднання в розділі FROM (JOIN ... ON).

3. Як виконати з'єднання таблиць із використанням розділу FROM?

Явне з'єднання записується безпосередньо в розділі FROM за допомогою оператора JOIN і умови ON, яка вказує, за якими стовпцями зіставляються рядки:

```
SELECT c.NAME_CM, o.AMOUNT_OD, o.DATE_OD
FROM Customers c
JOIN Orders o ON o.ID_CM = c.ID_CM;
```

Слово INNER можна не писати: JOIN без уточнення означає INNER JOIN. Для збереження рядків, яким не знайшлося пари, використовують LEFT JOIN, RIGHT JOIN або FULL JOIN, а для повного перехресного з'єднання – CROSS JOIN. Додаткові умови відбору записують у WHERE, а умови саме з'єднання – в ON:

```
SELECT p.publisher_name, COUNT(b.id_book) AS Кількість_книг
FROM Publisher p
LEFT JOIN Book b ON b.id_publisher = p.id_publisher
GROUP BY p.id_publisher, p.publisher_name;
```

Такий запис є наочнішим: одразу видно, які таблиці з'єднуються, за яким ключем і який тип з'єднання застосовано.

4. Які види з'єднань Ви знаєте? Наведіть їх характеристики.

У MS SQL Server використовують такі види з'єднань:

1) INNER JOIN – повертає лише ті рядки, для яких знайшлася пара в обох таблицях. Застосовується найчастіше, коли потрібні лише пов'язані записи (запити 1.2, 1.9).

2) LEFT OUTER JOIN (LEFT JOIN) – зберігає всі рядки лівої таблиці; якщо пари немає, стовпці правої таблиці заповнюються значенням NULL (запити 1.1, 1.17, 2.3).

3) RIGHT OUTER JOIN (RIGHT JOIN) – дзеркальний до LEFT JOIN: зберігає всі рядки правої таблиці, а відсутні значення лівої замінює на NULL.

4) FULL OUTER JOIN (FULL JOIN) – зберігає всі рядки обох таблиць; там, де пари немає, відповідні стовпці містять NULL.

5) CROSS JOIN – декартів добуток: кожен рядок першої таблиці поєднується з кожним рядком другої, умова ON не використовується.

6) SELF JOIN – з'єднання таблиці самої з собою; обов'язково використовуються два різні псевдоніми, наприклад для пошуку пар записів або ієрархічних зв'язків.

Крім того, розрізняють еквіз'єднання (умова ON містить рівність) і нееквіз'єднання (умова ON містить <, >, BETWEEN тощо).

5. У чому полягає різниця між INNER JOIN та OUTER JOIN?

INNER JOIN повертає тільки ті рядки, яким знайшлася відповідність в обох таблицях. Рядки без пари до результату не потрапляють. OUTER JOIN (LEFT, RIGHT або FULL) зберігає рядки однієї чи обох таблиць навіть тоді, коли пари для них немає; відсутні значення заповнюються NULL.

Різницю добре видно на прикладі запитів 1.13 і 1.17. Запит 1.13 з INNER JOIN показує лише замовників, які вже щось замовили:

```
-- INNER JOIN: лише замовники із замовленнями
SELECT c.ID_CM, c.NAME_CM, COUNT(*) AS Кількість
FROM Customers c
JOIN Orders o ON o.ID_CM = c.ID_CM
GROUP BY c.ID_CM, c.NAME_CM;
```

А запит 1.17 з LEFT JOIN додає до списку й замовника 2005, який ще не зробив жодного замовлення; для нього COUNT(o.ID_OD) дорівнює 0:

```
-- LEFT JOIN: усі замовники, зокрема без замовлень
SELECT c.ID_CM, c.NAME_CM, COUNT(o.ID_OD) AS Кількість
FROM Customers c
LEFT JOIN Orders o ON o.ID_CM = c.ID_CM
GROUP BY c.ID_CM, c.NAME_CM;
```

Отже, INNER JOIN звужує результат до пов'язаних рядків, а OUTER JOIN дозволяє зберегти повний перелік записів однієї таблиці й побачити, яким із них не вистачає пари.

6. Як здійснити об’єднання таблиць із використанням операції UNION?

Оператор UNION об’єднує результати двох або більше операторів SELECT в один набір рядків. Він має виконуватися за таких умов:

1) усі запити містять однакову кількість стовпців;

2) відповідні стовпці мають сумісні типи даних;

3) порядок стовпців у запитах однаковий, оскільки назви стовпців результату беруться з першого запиту;

4) ORDER BY записується один раз у кінці та застосовується до всього об’єднаного результату.

UNION автоматично вилучає дублікати, а UNION ALL залишає всі рядки, тому працює швидше. Приклад із цього звіту (запит 1.3):

```
SELECT NAME_SP AS Прізвище, N'продавець' AS Статус, CITY_SP AS Місто
FROM Salespeople
WHERE CITY_SP = N'Київ'
UNION
SELECT NAME_CM, N'замовник', CITY_CM
FROM Customers
WHERE CITY_CM = N'Київ'
ORDER BY Прізвище;
```

У результаті виходить один список із трьома стовпцями, де рядки з обох таблиць позначені відповідним статусом. Якщо потрібно зберегти всі входження, зокрема повторювані, замість UNION використовують UNION ALL.

Висновки:

Під час виконання лабораторної роботи № 5 було опановано створення складних багатотабличних запитів у MS SQL Server. Розглянуто явне та неявне з’єднання таблиць, види з’єднань INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN і CROSS JOIN, групування з агрегатними функціями COUNT, SUM, AVG, MIN і MAX, об’єднання результатів оператором UNION. На прикладі баз Sales111 і LibraryDB виконано 19 запитів за методичними вказівками та 8 запитів за індивідуальним варіантом (шість із JOIN і два з UNION). Отримані результати підтверджують, що складні запити дозволяють поєднувати дані з кількох таблиць, обчислювати підсумкові показники за групами та формувати єдині списки з різнорідних джерел. Окремо надано відповіді на всі шість контрольних питань з методичних вказівок.